

CFD WITH OPENSOURCE SOFTWARE

CHALMERS UNIVERSITY OF TECHNOLOGY (PROF. HÅKAN NILSSON)

Project Report:

Improvement of Lagrangian approach for multiphase flow

Developed in OpenFOAM-2.2.2

Author:
Matteo NOBILE

Peer reviewed by:
HAO CHEN
JELENA ANDRIC

Disclaimer: This is a student project work, done as part of a course where OpenFOAM and some other OpenSource software are introduced to the students. Any reader should be aware that it might not be free of errors. Still, it might be useful for someone who would like learn some details similar to the ones presented in the report and in the accompanying files. The material has gone through a review process. The role of the reviewer is to go through the tutorial and make sure that it works, that it is possible to follow, and to some extent correct the writing. The reviewer has no responsibility for the contents.

January 10, 2015

Contents

1	Theory	3
1.1	Injection Modeling	3
1.1.1	Injection Deactivation	3
1.1.2	Formulation of the new injection process	4
1.1.3	Structure of the class InjectionModel	5
1.2	Evaporation modelling	5
1.3	Calculation of drag coefficient	7
2	Implementation in OpenFOAM	9
2.1	Multiphase Lagrangian solver	9
2.2	Injection Modelling	12
2.2.1	Injection Deactivation	12
2.2.2	Formulation of the new injection process	12
2.2.3	Structure of the class InjectionModel	15
2.3	Evaporation modelling	15
2.4	Drag coefficient	16
2.4.1	Calculation of drag coefficient	16
2.4.2	Calculation of droplet deformation	17
2.5	New Lagrangian sub-libraries and solver	17
3	Case testing and results	21
3.1	Description of the base case	21
3.2	Description of the tested cases	23
3.3	Results	24
3.3.1	Case 1: Deactivation of the injection process	24
3.3.2	Case 2: Formulation of the new injection process	25
3.3.3	Case 3: Evaporation modeling	25
3.3.4	Case 4: Drag coefficient modeling	27
	Nomenclature	34
	Appendices	36
A	Evaporation Modeling	37
B	Drag Coefficient	45
C	New Lagrangian sub-libraries and solver	49

Introduction

The analysis of multiphase flow, and in this specific project three-phase flow (solid particles and liquid droplets in a gaseous carrier phase), is of great importance in several common industrial applications. The Lagrangian-Eulerian (LE) approach is a common and suitable method for dispersed multiphase flow wherein droplets or particles are represented in a Lagrangian reference frame while the carrier-phase flow field is represented in an Eulerian frame. In this project on OpenFOAM software, the Lagrangian library has been expanded for macro-scale modeling of three-phase flow.

The existing Lagrangian framework distinguishes between two-phase flow with specific liquid spray modeling (i.e. atomization, breakup, collision processes) and a general three-phase flow modeling in the sub-libraries `spray` and `intermediate` (`ReactingMultiphase` template class) respectively.

Moreover some existing Lagrangian submodels have been improved so that:

- a new evaporation model has been developed based on the classic theory [1].
- the calculation of drag coefficient for liquid droplet takes into account the effect of evaporation, deformation and internal circulation [2]

Thus this report is organized in the following way:

- the theoretical formulation of these new models is described in the first chapter. For the sake of brevity the existing formulations are not detailed here.
- the implementation of these models in two new Lagrangian sub-libraries is detailed in the second chapter
- in last chapter the existing tutorial "Aachen Bomb" is run by varying each modified model in order to compare simulation results between the existing and the modified one.

Chapter 1

Theory

This chapter describes the formulations of the modified Lagrangian submodels and it is organized by following the same steps of a typical Lagrangian solution (injection and then mass, momentum and energy conservation equations).

1.1 Injection Modeling

The existing model has been modified in order to:

- allow the deactivation of the injection process by avoiding a floating point error
- link together the input parameters for calculating the injected mass at each time step and match the total required mass to inject at the end of the injection process (particularly in case of low injected mass per time step)
- speed up the simulation timing of the injection process by reordering the solution steps

The structure of next subsections is organized according to these items.

1.1.1 Injection Deactivation

In order to understand the scenario, the method `averageParcelMass()` is called in two classes that belong to the `spray` Lagrangian sub-library:

- `SprayCloud` template class (constructor):

```
averageParcelMass_ = this->injectors().averageParcelMass();
```

- "KHRT" breakup model (member functions):

```
const scalar averageParcelMass = this->owner().averageParcelMass();
if (ms/averageParcelMass > msLimit_)
{...
```

while it is defined in the classes `InjectionModelList` as:

```
template<class CloudType>
Foam::scalar Foam::InjectionModelList<CloudType>::averageParcelMass()
{
    scalar mass = 0.0;
    scalar massTotal = 0.0;
    forAll(*this, i)
```

```

    {
        scalar mt = this->operator[](i).massTotal();
        mass += mt*this->operator[](i).averageParcelMass();
        massTotal += mt;
    }
    return mass/massTotal;
}

and InjectionModel as:

template<class CloudType>
Foam::scalar Foam::InjectionModel<CloudType>::averageParcelMass()
{
    label nTotal = 0.0;
    if (this->owner().solution().transient())
    {
        nTotal = parcelsToInject(0.0, timeEnd() - timeStart());
    }
    else
    {
        nTotal = parcelsToInject(0.0, 1.0);
    }
    return massTotal_/nTotal;
}

```

In the previous code it may be highlighted that when the injection is deactivated (refer to `NoInjection` class below) a floating point error occurs since in that case the method `parcelsToInject` return a zero value, as expected:

```

template<class CloudType>
Foam::label Foam::NoInjection<CloudType>::parcelsToInject
(
    const scalar,
    const scalar
)
{
    return 0;
}

```

1.1.2 Formulation of the new injection process

The main user-input variables for the injection process are:

- total required mass to inject m_{tot}
- parcels flow $\dot{n}$
- profile of the injection $p(t)$
- injection duration Δ

If neglecting other methods dedicated for particle tracking, the injection process, at each time step k , has been formulated as follow :

$$\begin{aligned}
 \text{parcel total } N_{tot} &= \dot{n}\Delta \\
 \text{volume total } V_{tot} &= \frac{m_{tot}}{\rho_0} \\
 \text{injection fraction } f_{inj} &= \frac{\int_k^k p(t)}{\int_0^\Delta p(t)}
 \end{aligned}$$

$$\begin{aligned}
&\text{injectable parcels } N_{avail} = \text{floor}(f_{inj} * N_{tot}) \\
&\text{delayed volume } V_{del}(k) = \begin{cases} 0 & \text{if } k = 0 \\ V_{del}(k-1) & \text{else} \end{cases} \\
&\text{initial injectable volume } V_{avail}(k) = V_{del} + f_{inj} V_{tot} \\
&\text{number of particles per parcel } i\text{-th of } N_{avail} \quad np_i = \frac{6V_{avail}}{(N_{avail} - i + 1)\pi d_i^3} \\
&\text{updated injectable volume } V_{avail}(k) = \begin{cases} V_{avail}(k) & \text{if } np_i < 1 \\ V_{avail}(k) - np_i \pi \frac{d_i^3}{6} & \text{if } np_i \geq 1 \end{cases}
\end{aligned}$$

The main differences with respect to the original formulation regard:

- the calculation of the number of particles per injected parcel, since they are injected by considering the current available parcels $(N_{avail} - i + 1)$ and not the total ones N_{avail}
- the total available parcels N_{avail} that are injected according to the injection profile at each time step

1.1.3 Structure of the class InjectionModel

The main method of the class `InjectionModel` is `inject(TrackData& td)`. The part of the original code which has been analyzed and modified is shown below:

```

if (!pPtr->move(td, dt))
{
    delete pPtr;
}
else
{
    if (pPtr->nParticle() >= 1.0)
    {
        td.cloud().addParticle(pPtr);
        massAdded += pPtr->nParticle()*pPtr->mass();
        parcelsAdded++;
    }
    else
    {
        delayedVolume += pPtr->nParticle()*pPtr->volume();
        delete pPtr;
    }
}

```

It highlights that, once the number of particles per parcel is evaluated, the main Lagrangian solution (with the method `move(td, dt)`) starts although the number of particles is less than one.

Then a check of each parcel deletes the ones with a number of particles higher or equal than one. In the latter cases the Lagrangian calculation would be useless, thus the order has been reversed so that the main Lagrangian calculation is run only if the number of particles is equal or higher than one. In this way a lot of time for running useless parcels is saved.

1.2 Evaporation modelling

The model for the evaporation process is based on Spalding's work [3]. The main assumptions are: negligible effect of droplet internal circulation and infinitely high diffusivity (the so called Rapid Mixing Model or Infinite Conductivity Model). For the sake of brevity, other hypothesis of the model and the integration of the conservation equations for gas and liquid phases are not reported here and can be found in References [4] and [5].

Since the evaluation of thermophysical properties of fluids has a strong impact on the simulation of the evaporation precess [6], a brief review of several works from literature has been taken into account in evaluating them.

Thus the thermophysical properties of the gas phase are calculated with the Mason and Saxena rule [7] for gaseous mixture and they have been evaluated at the film temperature T_{film} , which is defined using the 1/3 rule between the carrier and the droplet phase. In next formulas the subscripts c and d denote the carrier and the droplet phase, respectively:

$$T_{ref} = \frac{2T + T_c}{3} \quad (1.1)$$

$$gas\ phase = \left\{ \begin{array}{l} c_{p,c} = \sum_i^N Y_{c,i} c_{p,i,ref} \\ \lambda_c = \frac{\sum_i^N X_{c,i} \lambda_{c,i}}{\sum_j^N X_{c,j} \frac{(1 + \sqrt{\frac{\lambda_{c,i}}{\lambda_{c,j}}} * (\frac{PM_i}{PM_j})^{0.25})^2}{\sqrt{8(1 + \frac{PM_i}{PM_j})}}} \\ \mu_c = \frac{\sum_i^N X_{c,i} \mu_{c,i}}{\sum_j^N X_{c,j} \frac{(1 + \sqrt{\frac{\mu_{c,i}}{\mu_{c,j}}} * (\frac{PM_i}{PM_j})^{0.25})^2}{\sqrt{8(1 + \frac{PM_i}{PM_j})}}} \\ \rho_c = \frac{p_c}{RT_{ref}} \end{array} \right. \quad (1.2)$$

The properties of the evaporating specie in the vapor phase have been evaluated at film temperature while the composition at droplet surface has been calculated according to Raoult's law (liquid-vapor equilibrium):

$$vapor\ phase = \left\{ \begin{array}{l} p_{sat,i} = p_{sat}(T) \\ c_{p,i,vap} = c_p(T_{ref}) \\ \mu_{i,vap} = \mu(T_{ref}) \\ \lambda_{i,vap} = \lambda(T_{ref}) \\ D_{i,vap} = D_{i-air}(T_{ref}) \\ p_{i,vap} = X_{i,liq} p_{sat,i} \\ \rho_{i,vap} = \frac{p_{i,vap}}{RT_{ref}} \\ X_{i,vap} = \frac{X_{i,liq} p_{sat,i}}{p} \\ Y_{i,vap} = \frac{PM_i X_{i,vap}}{PM_i X_{i,vap} + (1 - X_{i,vap}) PM_c} \end{array} \right. \quad (1.3)$$

The film composition has been calculated using the 1/3 rule, like the temperature, while the other parameters have been evaluated with the ones of the gaseous and vapor phase. The thermal conductivity has been evaluated according to the Mason rule, while the viscosity has been considered with two alternative approaches of Zipperer and Wilke [7] respectively:

$$\text{film condition} = \left\{ \begin{array}{l}
Y_{ref} = \frac{2Y_{i_{vap}} + Y_c}{3} \\
c_{p,ref} = Y_{ref}c_{p,i_{vap}} + (1 - Y_{ref})c_{p,c} \\
\rho_{ref} = \left(\frac{Y_{ref}}{\rho_{i_{vap}}} + \frac{(1-Y_{ref})}{\rho_c} \right)^{-1} \\
\lambda_{ref} = \frac{X_{ref}\lambda_{i_{vap}}}{X_{ref} + (1 - X_{ref}) \frac{(1 + \sqrt{\frac{\lambda_{i_{vap}}}{\lambda_c} * (\frac{PM_c}{PM_j})^{0.25}})^2}{\sqrt{8(1 + \frac{PM_{i,vap}}{PM_c})}}} + \\
+ \frac{(1-X_{ref})\lambda_c}{(1 + \sqrt{\frac{\lambda_c}{\lambda_{i_{vap}}} * (\frac{PM_c}{PM_{vap}})^{0.25}})^2} \\
1 - X_{ref} + X_{ref} \frac{\sqrt{8(1 + \frac{PM_c}{PM_{vap}})}}{\sqrt{8(1 + \frac{PM_c}{PM_{vap}})}} \\
\mu_{ref} = \frac{X_{ref}\mu_{i_{vap}}}{X_{ref} + (1 - X_{ref}) \frac{(1 + \sqrt{\frac{\mu_{i_{vap}}}{\mu_c} * (\frac{PM_c}{PM_j})^{0.25}})^2}{\sqrt{8(1 + \frac{PM_{i,vap}}{PM_c})}}} + \\
+ \frac{(1-X_{ref})\mu_c}{(1 + \sqrt{\frac{\mu_c}{\mu_{i_{vap}}} * (\frac{PM_c}{PM_{vap}})^{0.25}})^2} \\
1 - X_{ref} + X_{ref} \frac{\sqrt{8(1 + \frac{PM_c}{PM_{vap}})}}{\sqrt{8(1 + \frac{PM_c}{PM_{vap}})}}
\end{array} \right. \quad (1.4)$$

Thus the mass and heat transfer parameters have been obtained as follows (the Grashof number can be included if natural convection is considered):

$$Re = \frac{\rho_c |u - u_c| d}{\mu_{ref}} \quad Sc = \frac{\mu_{ref}}{\rho_{ref} D_{i_{vap}}} \quad Gr = 9.81 d^3 \frac{\rho_c (T_c - T)}{T_c \mu_c} \quad Pr = \frac{\mu_{ref} c_{p,ref}}{\lambda_{ref}} \quad (1.5)$$

$$Nu_0 = 2.0009 + 0.514 \sqrt{\max(Re, \sqrt{Gr}) Pr^{\frac{1}{3}}} \quad Sh_0 = 2.0009 + 0.514 \sqrt{\max(Re, \sqrt{Gr}) Sc^{\frac{1}{3}}} \quad (1.6)$$

$$B_M = \frac{Y_{i_{vap}} - Y_c}{1 - Y_{i_{vap}}} \quad F_M = \frac{(1 + B_M)^{0.7}}{B_M} \log(1 + B_M) \quad Sh = 2 + \frac{Sh_0 - 2}{F_M} \quad (1.7)$$

$$\phi = \frac{c_{p,i_{vap}}}{c_{p,ref}} \frac{Sh}{Nu} \frac{Pr}{Sc} \quad B_T = (1 + B_M)^\phi - 1 \quad Nu = 2 + \frac{Sh_0 - 2}{F_T} \quad (1.8)$$

Finally, after the iterative cycle in (1.8) for evaluating the heat transfer number B_T , the evaporated mass and the heat transferred from the gas-phase to the droplet can be calculated as:

$$dM = \pi d Sh D_{i_{vap}} \rho_{ref} \log(1 + B_T) dt \quad (1.9)$$

$$dH = dM H_{vap} \quad (1.10)$$

1.3 Calculation of drag coefficient

In most of fluid-particle flows, the drag force gives the main contribution in the calculation of particle motion, i.e. its velocity and trajectory by solving the Newton's second law.

In order to model the forces acting on the interface of the two phases, one has to solve (either analytically or numerically) the distribution of the stresses on both sides of the interface particle-fluid. Since the mathematical problem of the time evolution of the fully coupled drop-fluid system

is very complicated, analytical solutions are possible only for very limiting cases [1]. Thus the expression of this force is evaluated as:

$$F_{drag} = \frac{\rho_{fl}}{2} \frac{\pi d^2}{4} C_d (u_c - u) \quad (1.11)$$

where C_d is the drag coefficient.

Since the drag force exerted by the fluid in particle is significant for particle motion, a great care has to be taken to choose the correct formulation for this coefficient. In case of a solid sphere moving through a fluid, for very small Reynolds numbers the coefficient can be determined analytically and this is referred to us as Stokes' solution. As the Reynolds number increases, vortex rings and their breakup in smaller structures occur and Stokes formulation fails to predict the drag. For Reynolds numbers greater than 20000 the drag coefficient changes very little and is almost equal to 4/9. Thus several correlations have been derived for the drag coefficient of spheres, each one for a particular range of the Reynolds number.

In this project the drag coefficient has been distinguished between:

- solid particles (or particles with more than one phase), that have been considered as rigid spheres, with two alternative formulations for $Re > 1000$ (the first alternative has been used in this project), as follow:

$$C_d = \begin{cases} \frac{24}{Re} (1 + \frac{1}{6} Re^{\frac{2}{3}}) [8] & Re < 1000 \\ \frac{24}{Re} (1 + 0.15 Re^{0.687}) [9] & Re < 1000 \\ 0.44 & Re > 1000 \end{cases} \quad (1.12)$$

- liquid droplets, for which both evaporation and deformation processes have been included. In literature the effect of evaporation process has been taken into account in two main ways. In 1983 Sirignano cited Yuen [10] that considered the vaporizing effect only by evaluating the viscosity in the boundary layer of the droplet (film) in the calculation of Reynolds number:

$$Re = \frac{\rho_c |u - u_c| d}{\mu_{ref}} \quad (1.13)$$

Then, as also cited in a successive work of Sirignano, Renksizbulut [11] included the Spalding heat transfer number B_T in the formulation.

At the same time droplet deformation may affect the motion of the drop by increasing the drag forces due to both the increased frontal area and drag coefficient. This effect has been considered with two alternative formulations of Liu [12] and Renksizbulut [11]. Finally the formulations considered for the calculation of the drag coefficient for a liquid droplet are:

$$C_d = \begin{cases} \frac{24}{Re} (1 + 0.13 Re^{0.82 - \log Re}) & Re \leq 20 \\ \frac{24}{Re} (1 + B_T)^{-0.2} (1 + 0.2 Re^{0.63}) (1 + 0.06 Re^{-0.12} We^{1.4}) & 20 < Re < 300 \\ \frac{24}{Re} (1 + \frac{1}{6} Re^{\frac{2}{3}}) (1 + 2.632y) & Re \geq 300 \end{cases} \quad (1.14)$$

where y is the parameter that evaluates droplet deviation from its spherical shape.

Chapter 2

Implementation in OpenFOAM

This chapter describes the implementation of the submodels and formulations previously discussed. The structure of the chapter follows the same organization of the previous one, although the first section is dedicated to the integration of the multiphase Lagrangian capability within the existing framework for the analysis of a monophasic (liquid) spray.

2.1 Multiphase Lagrangian solver

The original Lagrangian sub-library for simulation of liquid droplets has been extended in order to include other phases like the solid one. This is of great importance for analyzing phase change phenomena in several applications, such as spray metal forming and urea decomposition process in Diesel Aftertreatment Systems.

Specifically the typical liquid spray submodels such as atomization and breakup have been taken into account only for liquid single-phase droplets.

From a practical point of view, this integration has been realized by adding new variables to the parcel such as total solid and gas mass fraction and single specie mass fraction. Moreover models for phase change processes such as surface reaction and devolatilization have been implemented.

These modifications have been realized in the template classes `SprayCloud` and `SprayParcel` and for the sake of brevity only a part of them is shown below. The comparison between the original and the modified class may be obtained with the `git diff` command (after installing the `git` package) and is shown below.

SprayParcel.C

```
--- a/$WM_PROJECT_USER_DIR/src/lagrangian/spray/parcels/Templates/  
    /SprayParcel/SprayParcel.C  
+++ b/$WM_PROJECT_USER_DIR/src/lagrangian/spray/parcels/Templates/  
    /SprayParcel/SprayParcel.C  
#include "SprayParcel.H"  
#include "CompositionModel.H"  
#include "AtomizationModel.H"  
    ....  
-template<class ParcelType>  
-template<class TrackData>  
-Foam::scalar Foam::SprayParcel<ParcelType>::CpEff  
-(  
-    TrackData& td,  
-    const scalar p,  
-    const scalar T,  
-    const label idG,
```

```

-     const label idL,
-     const label idS
-) const
-{
-     return
-         this->Y_[GAS]*td.cloud().composition().Cp(idG, YGas_, p, T)
-         + this->Y_[LIQ]*td.cloud().composition().Cp(idL, YLiquid_, p, T)
-         + this->Y_[SLD]*td.cloud().composition().Cp(idS, YSolid_, p, T);
-}

.....

// * * * * * Protected Member Functions * * * * * //
.....

-     // 6. Devolatilisation
-     newCalcDevolatilisation
-     (
-         td,
-         dt,
-         this->age_,
-         Ts,
-         d0,
-         T0,
-         mass0,
-         this->mass0_,
-         YMix[GAS]*YGas_,
-         YMix[LIQ]*YLiquid_,
-         YMix[SLD]*YSolid_,
-         canCombust_,
-         dMassDV,
-         Sh,
-         Ne,
-         NCpW,
-         Cs
-     );
-
-     // 7. Surface reactions
-     newCalcSurfaceReactions
-     (
-         td,
-         dt,
-         cellI,
-         d0,
-         T0,
-         mass0,
-         canCombust_,
-         Ne,
-         YMix,
-         YGas_,
-         YLiquid_,
-         YSolid_,
-         dMassSRGas,
-         dMassSRLiquid,

```

```

-         dMassSRSolid,
-         dMassSRCarrier,
-         Sh,
-         dhsTrans
-     );
-
-     // 8. Update the parcel mass and mass fractions due to phase change
-     scalarField dMassGas(dMassDV + dMassSRGas);
-     scalarField dMassLiquid(dMassPC + dMassSRLiquid);
-     scalarField dMassSolid(dMassSRSolid);
-     scalar mass1 = updateMassFractions(mass0, dMassGas, dMassLiquid, dMassSolid);

    // 14. Calc breakup
-     if ((YMix[LIQ] - 1.0) < SMALL)
-     {
-         if (td.keepParticle)
-         {
-             if (liquidCore() > 0.5)
-             {
-                 calcAtomization(td, dt, cellI);
-                 scalar d2 = this->d();
-                 this->nParticle() *= pow3(d1/d2);
-             }
-             else
-             {
-                 calcBreakup(td, dt, cellI);
-             }
-         }
-     }

// * * * * * Constructors * * * * *

template<class ParcelType>
Foam::SprayParcel<ParcelType>::SprayParcel(const SprayParcel<ParcelType>& p)
:
    ParcelType(p),
    d0_(p.d0_),
    position0_(p.position0_),
    liquidCore_(p.liquidCore_),
-     Reynolds_(p.Reynolds_),
-     Weber_(p.Weber_),
-     om2_(p.om2_),
    KHindex_(p.KHindex_),
    y_(p.y_),
    yDot_(p.yDot_),
    ms_(p.ms_),
    injector_(p.injector_),
    tMom_(p.tMom_),
    user_(p.user_),
-     Bt_(p.Bt_),
-     YGas_(p.YGas_),
-     YLiquid_(p.YLiquid_),
-     YSolid_(p.YSolid_),
-     canCombust_(p.canCombust_)

```

```
{}
```

2.2 Injection Modelling

2.2.1 Injection Deactivation

The possibility of deactivating the injection process has been implemented by adding an `if` statement within the method `averageParcelMass()` in the class `InjectionModelList`. Specifically, the flag `active` has been used since it is set to `false` in the `NoInjection` class. So, when the injection is deactivated, the method return a value equal to 1.0 (it may be a generic value).

```
template<class CloudType>
Foam::scalar Foam::InjectionModelList<CloudType>::averageParcelMass()
{
    scalar mass = 0.0;
    scalar massTotal = 0.0;
    forAll(*this, i)
    {
        if (!this->operator[](i).active())
        {
            mass = 1.0;
            massTotal = 1.0;
        }
        else
        {
            scalar mt = this->operator[](i).massTotal();
            mass += mt*this->operator[](i).averageParcelMass();
            massTotal += mt;
        }
    }
    return mass/massTotal;
}
```

2.2.2 Formulation of the new injection process

The new formulation has been implemented by both modifying the existing main function `inject(TrackData& td)` for transient injection process and several methods that are called in this function.

Since these modifications included adding new input variables for some methods shared with the main function `injectSteadyState(TrackData& td, const scalar trackTime)` for steady-state injection, they have been also applied in it.

The different formulation for the injection process can be understood by comparing the existing and the new method `inject(TrackData& td)`, as shown below, and by referring to the formulation in 1.1.2.

Thus the variable `scalar delayedVolume_` refers to V_{del} and it is used by `scalar Vdisp` as described in V_{avail} , `scalar density` refers to ρ_0 and it is read from dictionary (`constantProperties`). Moreover it can be seen that some secondary methods (such as `setNumberOfParticles`, `postInjectCheck`) have more input variables since they have been also modified as shown later.

```
--- a/$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/.../InjectionModel.C
+++ b/$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/.../InjectionModel.C

    scalar massAdded = 0.0;
    label newParcels = 0;
    scalar newVolumeFraction = 0.0;
```

```

-   scalar deleted = 0;

    if (prepareForNextTimeStep(time, newParcels, newVolumeFraction))
    {
+       scalar delayedVolume = 0;
        .....
-       scalar density = cloud.constProps().rho0(); //Set density equal to rho0
-       scalar Vdisp = delayedVolume_ + newVolumeFraction*massTotal_/density;
        ....
        // Introduce new parcels
        for (label parcelI = 0; parcelI < newParcels; parcelI++)
        {
            .....
            pPtr->nParticle() = setNumberOfParticles(
-                parcelI,
-                newParcels,
-                Vdisp,
+                newParcels,
+                newVolumeFraction,
+                pPtr->d(),
-                density
+                pPtr->rho());
            .....
+            if (pPtr->nParticle() >= 1.0)
            {
                td.cloud().addParticle(pPtr);
-                massAdded += pPtr->nParticle()*pPtr->volume()*density;
-                Vdisp -= pPtr->nParticle()*pPtr->volume();
+                massAdded += pPtr->nParticle()*pPtr->mass();
                parcelsAdded++;
            }
-            else
            {
-                delete pPtr;
-                deleted++;
+            else
            {
+                delayedVolume += pPtr->nParticle()*pPtr->volume();
+                delete pPtr;
                .....
            }
-            delayedVolume_ = Vdisp;
+            delayedVolume_ = delayedVolume;
        }
-        postInjectCheck(parcelsAdded, massAdded, deleted, newParcels,
            newVolumeFraction);
+        postInjectCheck(parcelsAdded, massAdded);

```

The method `parcelsToInject(const scalar time0, const scalar time1)`, defined in the injection submodel class `ConeNozzleInjection`, has been modified according to the calculation of N_{avail} in Section 1.1.2. The comparison between the original and the new method is shown below:

```

--- a/$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/.../ConeNozzleInjection.C
+++ b/$WM_PROJECT_DIR/src/lagrangian/intermediate/.../ConeNozzleInjection.C

```

```

        if ((time0 >= 0.0) && (time0 < duration_))
        {
-       scalar nTOT = floor(duration_*parcelsPerSecond_);
-       scalar volFract = flowRateProfile_.integrate(time0, time1)/this->volumeTotal();
-       return floor(nTOT*volFract);
+       return floor((time1 - time0)*parcelsPerSecond_);
        }
    else

```

In the method `setNumberOfParticles` the number of particles per parcels (np_i in Section 1.1.2) has been calculated with the density specified in the dictionary (see the definition of the variable `density` in the `inject` method).

```

--- a/$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/.../InjectionModel.C
+++ b/$WM_PROJECT_DIR/src/lagrangian/intermediate/.../InjectionModel.C

```

```

template<class CloudType>
Foam::scalar Foam::InjectionModel<CloudType>::setNumberOfParticles
(
-   const label iter,
    const label parcels,
-   const scalar Vdispon,
+   const scalar volumeFraction,
    const scalar diameter,
-   const scalar density
+   const scalar rho
)
{
    scalar nP = 0.0;
    .....
    case pbMass:
    {
        scalar volumep = pi/6.0*pow3(diameter);
-       nP = floor(Vdispon/((parcels-iter)*volumep));
+       scalar volumeTot = massTotal_/rho;
+       nP = (volumeFraction*volumeTot + delayedVolume_)/(parcels*volumep);
        break;
    }
}

```

Finally, for post-processing purposes, some injection parameters, such as the fraction of the injection and the injectable parcels during each time step, have been returned as output by adding new `Info` statements in the method `postInjectCheck`:

```

--- a/$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/.../InjectionModel.C
+++ b/$WM_PROJECT_DIR/src/lagrangian/intermediate/.../InjectionModel.C

```

```

void Foam::InjectionModel<CloudType>::postInjectCheck
(
    const label parcelsAdded,
-   const scalar massAdded,
-   const scalar deleted,
-   const label newParcels,
-   const scalar newVolumeFraction
+   const scalar massAdded
)

```

```

{
    const label allParcelsAdded = returnReduce(parcelsAdded, sumOp<label>());
-   const scalar allParcelsDeleted = returnReduce(deleted, sumOp<scalar>());

+   if (allParcelsAdded > 0)
+   {
        Info<< nl
            <<"--> Cloud: " << this->owner().name()
            << " injector: " << this->modelName() << nl
-           << "    Added " << allParcelsAdded << " new parcels of "
                << newParcels << nl
-           << "    Deleted " << allParcelsDeleted << " parcels because
                parcel mass too low" << nl
-           << "    Fraction of injection: " << newVolumeFraction << nl << endl;
+           << "    Added " << allParcelsAdded << " new parcels" << nl << endl;
+       }
        // Increment total number of parcels added

```

2.2.3 Structure of the class InjectionModel

As discussed in the previous chapter, the order of the if statements `move(td, dt)` and `nParticle()` has been easily reversed in the `InjectionModel` class:

```

if (pPtr->nParticle() >= 1.0)
{
    if (!pPtr->move(td, dt))
    {
        delete pPtr;
    }
    else
    {
        td.cloud().addParticle(pPtr);
        massAdded += pPtr->nParticle()*pPtr->volume()*density;
        Vdisp -= pPtr->nParticle()*pPtr->volume(); //ADDED
        parcelsAdded++;
    }
}
else
{
    delete pPtr;
    deleted++;
}

```

2.3 Evaporation modelling

The implementation of the new formulation for droplet evaporation process follows the same steps described in the previous section:

- creation of the new class `NewLiquidEvaporation`
- add the Spalding heat transfer number B_t in the constructor of template class `SprayParcel` (see section 2.1). This is done for speeding up the convergence of iterative cycle in Eq. 1.8. Indeed at each time step the converged value of B_t at previous time step is used as first value in the iteration.

- update the input variables of the main method `calculate` for solving droplet phase change

The new class is defined and described in the files `.H` and `.C`, located in `$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/submodels/Reacting/PhaseChangeModel/.` and shown in Appendix A (the `.C` file has been reduced by neglecting the models similar to other classes for evaporation modeling).

The comparison of the main method `calculate` for evaporation modeling with the original one highlights that the implementation of the new formulation requires to both change the definition of some existing input variables and to create the new one.

```
--- a/$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/.../NewLiquidEvaporation.C
+++ b/$WM_PROJECT_DIR/src/lagrangian/intermediate/.../LiquidEvaporation.C
```

```
template<class CloudType>
- void Foam::NewLiquidEvaporation<CloudType>::calculate
+ void Foam::LiquidEvaporation<CloudType>::calculate
(
    const scalar dt,
    const label cellI,
-   scalar& Re,
+   const scalar Re,
    const scalar Pr,
    const scalar d,
    const scalar nu,
    const scalar T,
    const scalar Ts,
    const scalar pc,
    const scalar Tc,
    const scalarField& Yl,
    scalarField& dMassPC,
-   scalar& bt,
-   const scalar mu
) const
```

For example the Reynolds number has to be defined as a non-constant pointer since it has to be updated by taking into account film viscosity μ_{ref} (not the carrier viscosity μ_c), according to definition in Eq. (1.13). Instead the scalar pointer B_T has been introduced to speed up the iteration cycle as previously discussed.

The main obstacle for implementing such class has regarded the conflicts generated in the definition of this method. Indeed it is defined as virtual method in the class `PhaseChangeModel` with lower input variables, it is reimplemented in other classes for phase change modeling and it is called in another function. Thus in order to avoid conflict in the compilation of the library, the input variables of the method have been updated in both the classes that model the evaporation process and the ones that call it for calculating phase change (e.g. `calcPhaseChange` in classes `ReactingParcel` and `ReactingMultiphaseParcel`). These classes can be found by typing the command `grep -rnw $WM_PROJECT_DIR/src -e "phaseChange"` in the terminal.

2.4 Drag coefficient

2.4.1 Calculation of drag coefficient

The new formulation for calculating the drag coefficient has been implemented in the new class `DeformedSphereDrag`. This class has been added in the `Drag` directory located in `$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/submodels/Kinematic/ParticleForces/`
`/Drag/`. The code of the `.H` and `.C` files is shown in Appendix B.

Again, as discussed in the previous section, the method `calcCoupled` is defined as virtual method in the class `ParticleForceList` with lower input variables. Indeed the original method is characterized as:

```
calcCoupled
(
    const typename CloudType::parcelType & p,
    const scalar dt,
    const scalar mass,
    const scalar Re,
    const scalar muc
)
```

and it is reimplemented in all the classes that evaluate the Lagrangian forces that depend on particle and carrier velocities (U_p and U respectively). These force terms, called Sp , are due to: pressure gradient, virtual mass and lift.

Moreover the method `calcCoupled` is called in the function `calcVelocity`, within the template class `KinematicParcel`, for solving particle velocity (according to the second Newton's law). Indeed the latter function `calcVelocity` is dictated to build the velocity-based Lagrangian force terms described before.

So the new variables have been added in all these classes. The latter can be found by typing the command `grep -rnw $WM_PROJECT_DIR/src -e "calcCoupled"`.

2.4.2 Calculation of droplet deformation

Since the drag coefficient has been formulated in Eq. (1.14) even by taking into account the effect of liquid droplet deformation, the existing method for the evaluation of the droplet distortion y has been checked in `$WM_PROJECT_DIR/src/lagrangian/spray/submodels/BreakupModel.H`.

The solution of the droplet oscillation equation is activated according to the status of the flag `solveOscillationEq`. It is initialized as `false` in the constructor of the class `BreakupModel` and is reimplemented as `true` only when the TAB breakup model is selected. For this reason, since the equation has to be solved regardless of the chosen breakup model, the constructor has been changed to `true` as follows:

```
--- a/$WM_PROJECT_USER_DIR/src/lagrangian/spray/.../BreakupModel.H
+++ b/$WM_PROJECT_DIR/src/lagrangian/spray/.../BreakupModel.H

//- Construct from dictionary
BreakupModel
(
    const dictionary& dict,
    CloudType& owner,
    const word& type,
-    bool solveOscillationEq = true
+    bool solveOscillationEq = false
);
```

2.5 New Lagrangian sub-libraries and solver

The formulations described in the first chapter have been implemented in classes located in the Lagrangian sub-libraries `intermediate` and `spray`. For this reason they have been duplicated in order to create new sub-libraries. The steps realized for this purpose are described below:

- create a new directory for Lagrangian library (if never done before) in the user `src`:
`mkdir -p $WM_PROJECT_USER_DIR/src/lagrangian`

- copy the original sub-libraries `intermediate` and `spray` in the new directory:

```
cp -r $WM_PROJECT_DIR/src/lagrangian/intermediate $WM_PROJECT_USER_DIR/src/  
/lagrangian  
cp -r $WM_PROJECT_DIR/src/lagrangian/spray $WM_PROJECT_USER_DIR/src/  
/lagrangian
```
- in the new directory, create `/Allwmake` and `/Allwclean` files for compiling the new sub-libraries. They are provided in Appendix C. An alternative is to copy the original `/Allwmake` file:

```
cp $WM_PROJECT_DIR/src/lagrangian/Allwmake $WM_PROJECT_USER_DIR/src/lagrangian  
and remove the lines not related to the new intermediate and spray sublibraries. Then copy  
the latter file as /Allwclean:  
cd $WM_PROJECT_USER_DIR/src/lagrangian  
cp Allwmake Allwclean  
and replace wmake with wclean:  
sed -i s/wmake/wclean/g Allwclean
```
- clean the new Lagrangian library

```
cd $WM_PROJECT_USER_DIR/src/lagrangian  
./Allwclean
```
- modify the destination and the name of the `.so` files of the new sub-libraries `intermediate` and `spray`, so that the new `$WM_PROJECT_USER_DIR/src/lagrangian/*/Make/files` (the `*` means `intermediate` and `spray`) is as follows:

```
LIB = $(FOAM_USER_LIBBIN)/libbnlagrangian*
```
- since the `spray` sub-library refers to the `intermediate` one, change in `$WM_PROJECT_USER_DIR/src/lagrangian/spray/Make/options` file the location and the link of the original `intermediate` sub-library with the new one:

```
EXE_INC = \  
....  
-I../intermediate/lnInclude \  
....  
LIB_LIBS = \  
....  
-L$(FOAM_USER_LIBBIN)\  
-lbnlagrangianIntermediate \  
....
```
- compile the new Lagrangian library:

```
./Allwmake
```
- insert and replace the new and the modified files in the sub-libraries
- compile again and check if it gets errors

In order to make the solver use the new Lagrangian sub-libraries, two alternatives can be used:

- maintain the original solver (`sprayFoam` in this project) and create a dynamic link between this solver and the new sub-libraries. This can be realized by adding them in the `controlDict` file of the case that has to be simulated

- create a new solver and link the new sub-libraries in the **Make** directory for compiling them together.

In this report the second option has been used since the presented work is included within a more huge project that includes the modification of the existing solver **sprayFoam** (e.g by replacing the combustion model with a generic chemistry model). So the next steps have to be followed:

- copy an original solver that is similar to your applications (in this project it is **sprayFoam**) in the user's solvers directory and rename the directory according to the name of the new solver (e.g. **ureaFoam**):

```
cp -r $WM_PROJECT_DIR/applications/solvers/lagrangian/sprayFoam
$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam
```

- clean the new solver

```
cd $WM_PROJECT_USER_DIR/applications/solvers/ureaFoam
wclean
```

- rename the new solver:

```
mv $WM_PROJECT_USER_DIR/applications/solvers/ureaFoam/sprayFoam.C
$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam/ureaFoam.C
sed -i s/sprayFoam/ureaFoam/g ureaFoam.C
```

- delete the **Allwmake** and the **sprayEngineFoam** sub-solver:

```
rm $WM_PROJECT_USER_DIR/applications/solvers/ureaFoam/Allwmake
rm -r $WM_PROJECT_USER_DIR/applications/solvers/ureaFoam/sprayEngineFoam
```

- modify the destination and the name of the executable file of the new solver so that the new **\$WM_PROJECT_USER_DIR/applications/solvers/lagrangian/ureaFoam/Make/files** is as follows:

```
ureaFoam.C
EXE = $(FOAM_USER_APPBIN)/ureaFoam
```

- copy the missing files for species and energy conservation equations from the solver **reactingParcelFoam** (found in the **ureaFoam/Make/options** file)

```
cp -r $WM_PROJECT_DIR/applications/solvers/lagrangian/reactingParcelFoam/YEqn.H
$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam
cp -r $WM_PROJECT_DIR/applications/solvers/lagrangian/reactingParcelFoam/EEqn.H
$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam
```

- in **\$WM_PROJECT_USER_DIR/src/lagrangian/spray/Make/options** file delete the location of **reactingParcelFoam**. Then change the location and the link of the original **intermediate** and **spray** sub-libraries as follows:

```
EXE_INC = \
....
-I$(WM_PROJECT_USER_DIR)/src/lagrangian/intermediate/lnInclude \
-I$(WM_PROJECT_USER_DIR)/src/lagrangian/spray/lnInclude \
....
EXE_LIBS = \
....
```

```
-L$(FOAM_USER_LIBBIN)\
-lbnlagrangianIntermediate \
-lbnlagrangianSpray \
....
```

- compile the new solver:

```
wmake
```

- check if the new solver is linked to the new sub-libraries:

```
ldd 'which ureaFoam'.
```

The correct output should be:

```
linux-vdso.so.1 =>
....
liblagrangian.so =>
$WM_PROJECT_DIR/platforms/linux64GccDP0pt/lib/liblagrangian.so
libbnlagrangianIntermediate.so =>
$WM_PROJECT_USER_DIR/platforms/linux64GccDP0pt/lib/libbnlagrangianIntermediate.so
libbnlagrangianSpray.so =>
$(WM_PROJECT_USER_DIR)/platforms/linux64GccDP0pt/lib/libbnlagrangianSpray.so
....
```

Chapter 3

Case testing and results

In this chapter the capability of new sub-libraries are tested and analyzed. Thus next sections describe the reference case, the tested ones and finally their results.

3.1 Description of the base case

The solver `ureaFoam` has been tested several times on the "Aachen Bomb" tutorial case by varying the new implemented models.

In order to reduce time for simulations, the total required injected mass and duration has been scaled by a factor 0.1 (i.e by maintaining the same mass flow rate of the original case). This modification has been realized in the file `SprayCloudProperties` located in the case sub-directory `constant` shown below (only `massTotal`, `duration` and the temporal `flowRateProfile` have been modified):

```
....
subModels
{
    particleForces
    {
        sphereDrag;
    }

    injectionModels
    {
        model1
        {
            type                coneNozzleInjection;
            SOI                  0;
            massTotal            6.0e-7;
            parcelBasisType      mass;
            injectionMethod      disc;
            flowType              flowRateAndDischarge;
            outerDiameter        1.9e-4;
            innerDiameter        0;
            duration              1.25e-4;
            position              ( 0 0.0995 0 );
            direction             ( 0 -1 0 );
            parcelsPerSecond      2.0e7;
            flowRateProfile       table
            (
                ( 0 0.1272 )
            )
        }
    }
}
```

```
( 4.16667E-006 6.1634 )
( 8.33333E-006 9.4778 )
( 0.0000125 9.5806 )
( 1.66667E-005 9.4184 )
( 2.08333E-005 9.0926 )
( 0.000025 8.7011 )
( 2.91667E-005 8.2239 )
( 3.33333E-005 8.0401 )
( 0.0000375 8.845 )
( 4.16667E-005 8.9174 )
( 4.58333E-005 8.8688 )
( 0.00005 8.8882 )
( 5.41667E-005 8.6923 )
( 5.83333E-005 8.0014 )
( 0.0000625 7.2582 )
( 6.66667E-005 7.2757 )
( 7.08333E-005 6.968 )
( 0.000075 6.7608 )
( 7.91667E-005 6.6502 )
( 8.33333E-005 6.7695 )
( 0.0000875 5.5774 )
( 9.16667E-005 4.8649 )
( 9.58333E-005 5.0805 )
( 0.0001 4.9547 )
( 0.000104167 4.5613 )
( 0.000108333 4.4536 )
( 0.0001125 5.2651 )
( 0.000116667 5.256 )
( 0.000120833 5.1737 )
( 0.000125 3.9213 )
( 0.0001251 0 )
( 1000 0 )
);
....
dispersionModel none;

patchInteractionModel standardWallInteraction;

heatTransferModel RanzMarshall;

compositionModel singlePhaseMixture;

phaseChangeModel none;

surfaceFilmModel none;

atomizationModel none;

breakupModel none;

stochasticCollisionModel none;

radiation off;
```

```

    standardWallInteractionCoeffs
    {
        type            rebound;
    }
    ....
}

```

Then this reference configuration has been modified according to the test cases described in next section.

Moreover combustion process has been deactivated since it is an "external" factor that can influence the analysis of the results. Thus the flags for chemistry and combustion processes have been set to **false** in the files **chemistryProperties** and **combustionProperties** in **constant**.

The other files in this sub-directory (**radiationProperties**, **g**, **thermophysicalProperties**, **turbulenceProperties**, **RASProperties**), as well as the boundary and initial conditions specified in the case sub-directory 0 (**epsilon**, **k**, **N2**, **O2**, **p**, **T** and **U** files), have been maintained as the original ones.

3.2 Description of the tested cases

After their implementation in OpenFOAM, the new formulations have been tested in four different cases. Each one focuses on a specific formulation and the modifications of the base file **SprayCloudProperties** are described below:

1. deactivation of the injection process.
This is done by modifying the injection **type none**; and running both **sprayFoam** (which is linked to the original library) and the new **ureaFoam** solvers
2. reduction of the total required mass to inject by modifying **massTotal 6.0e-8**; and running both **sprayFoam** and **ureaFoam** solvers
3. evaporation modeling by modifying **phaseChangeModel liquidEvaporationBoil**; and **phaseChangeModel newLiquidEvaporation**; for using the original and the new evaporation formulation, respectively. In this case only **ureaFoam** solver is run
4. drag coefficient modeling by modifying **particleForces {sphereDrag;}** and **particleForces {deformedSphereDrag;}** and using both new injection and new evaporation formulations

Thus, for simulating each case the following steps have been done:

- create the mesh of the case:

```
cd $WM_PROJECT_USER_DIR/run/"case"
blockMesh
```
- decompose the case for parallel simulation (if needed) according to the decomposition properties specified in the **decomposeParDict** file located in the case sub-directory **system**

```
decomposePar
```
- run the parallel simulation in background and write the output in a **log** file. In order to compare the results of the original and the Lagrangian sub-libraries, the following name "**solver**" has been changed for simulating the original and the new solver **sprayFoam** and **ureaFoam**, respectively:

```
foamJob -parallel "solver"
```

The following table summarizes the cases previously described.

CASE	INJECTION MODEL	EVAPORATION MODEL	DRAW COEFFICIENT MODEL
1	None	None	Sphere
2	Mass Total=6e-8 kg	None	Sphere
3	Mass Total=6e-7 kg	Original — New	Sphere
4	Mass Total=6e-7 kg	New	Deformed — Sphere

3.3 Results

For post-processing purposes, for each case "case":

- a file `case.OpenFOAM` has been created for comparing in ParaView results from different models:
`cd $WM_PROJECT_USER_DIR/run/"case"`
`touch case.OpenFOAM`
- a `foamLog.db` database file has been created by modifying the original one in order to extract some useful data from the `log` file with the `foamLog log` utility.
- a `.gplt` file has been created for plotting and comparing in gnuplot the previous data. As an alternative the same data can be post-processed using Matlab.

3.3.1 Case 1: Deactivation of the injection process

It is easy to demonstrate the benefit of the new implementation for deactivating the injection process. Firstly a simulation with the original Lagrangian sub-libraries has been run. The simulation stops since the floating point error occurs when evaluating the average parcel mass, as it can be seen in the log file below:

```
....
Selecting BreakupModel none
Selecting StochasticCollisionModel none
[2] #0 [3] [0] #0 #0 [1] #0 Foam::error::printStack(Foam::Ostream&)Foam::error::
printStack(Foam::Ostream&)Foam::error::printStack(Foam::Ostream&)Foam::error::
printStack(Foam::Ostream&) in in "/home/matteo/OpenFOAM/OpenFOAM-2.2.2/platforms/
/linux64GccDP0pt/lib/libOpenFOAM.so"
[3] #1 "/hoFoam::sigFpe::sigHandler(int) in "/home/matteo/OpenFOAM/OpenFOAM-2.2.2/
/platforms/linux64GccDP0pt/lib/libOpenFOAM.so"
[1] #1 Foam::sigFpe::sigHandler(int)me/matteo/OpenFOAM/OpenFOAM-2.2.2/platforms/
/linux64GccDP0pt/lib/libOpenFOAM.so"
....
```

Then the new implementation, described in Section 2.2.1, has been tested. In the new log file it can be seen that the average parcel mass is set to 1.0, as expected:

```
....
Selecting BreakupModel none
Selecting StochasticCollisionModel none
Selecting devolatilisation model none
Selecting surface reaction model none
Average parcel mass: 1
```

```
Selecting radiationModel none
.....
```

3.3.2 Case 2: Formulation of the new injection process

The main advantage of the new formulation is the possibility of matching the total required mass to inject at the end of a simulation. Several cases have been investigated by varying the parcel flow rate (by increasing and decreasing the base flow) and the mass flow rate.

This advantage has been demonstrated when the total mass to inject has been reduced, as explained in the second test case of Section 3.2. Indeed the original formulation does not allow to obtain the required mass, as it is highlighted in the injected mass evolution profile (the curve in red) of Figure 3.1.

Instead the new formulation (the blue injection profile) allows, at the end of the injection process, to match the input total mass to inject `massTotal` (green curve).

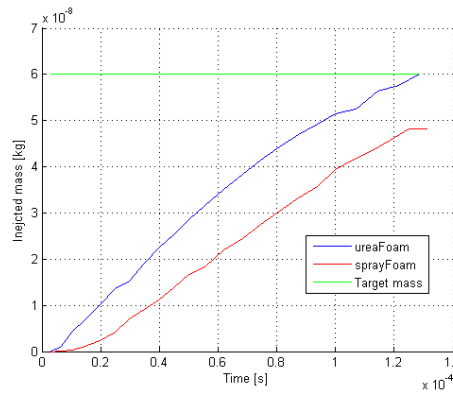


Figure 3.1: Comparison of the injected mass profiles between the original and the new formulation for injection process

3.3.3 Case 3: Evaporation modeling

The different formulation between the original and the new evaporation models can be analyzed by comparing the profiles of both gas and particles minimum temperature, total parcel and vaporized mass over time shown in Figure 3.2. Each parameter has an evolution coherent with an initial higher vaporization rate with the new evaporation model. However it can be also highlighted the importance of analyzing the long-term evolution of the evaporation process, since with the new model the rate decreases after 1ms and becomes lower than the one obtained with the original formulation.

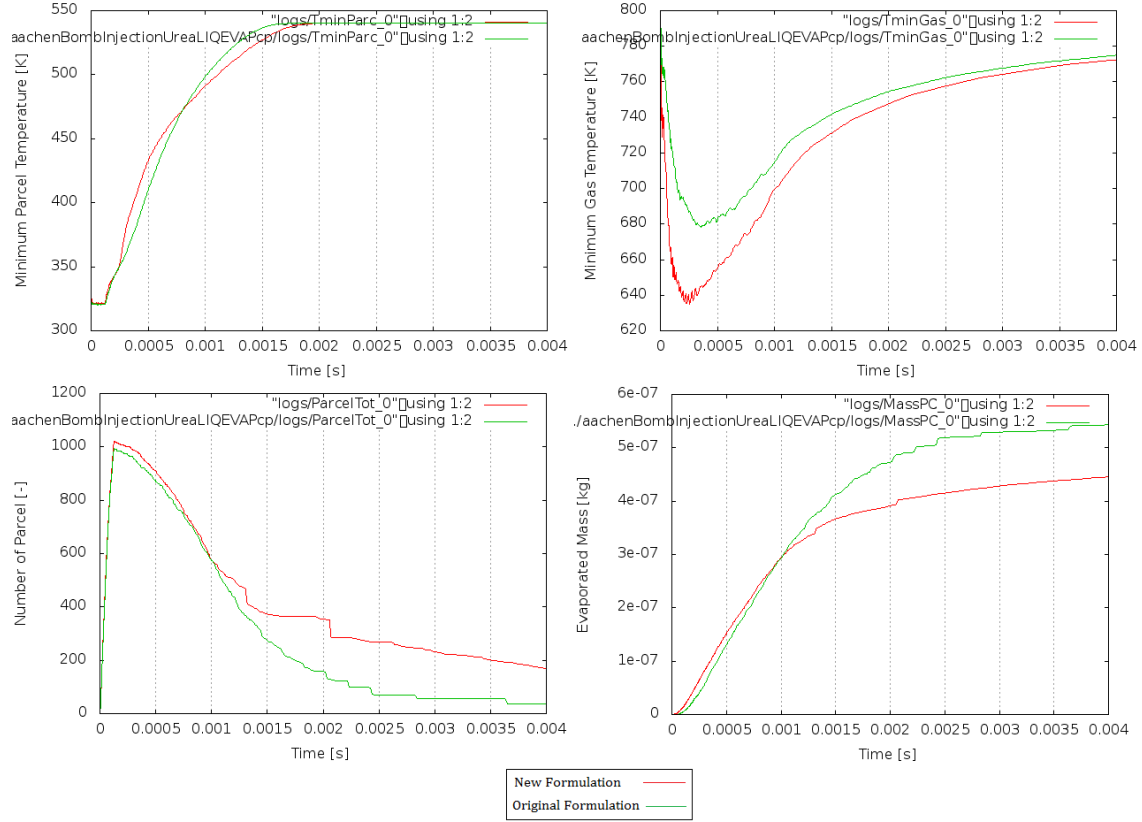


Figure 3.2: Comparison of profiles of minimum temperature of the phases, total parcel and vaporized mass in the system between the two evaporation models

A comparison of the field of the vapor specie is shown in Figure 3.3 and a different distribution can be noticed as soon as the process evolves.

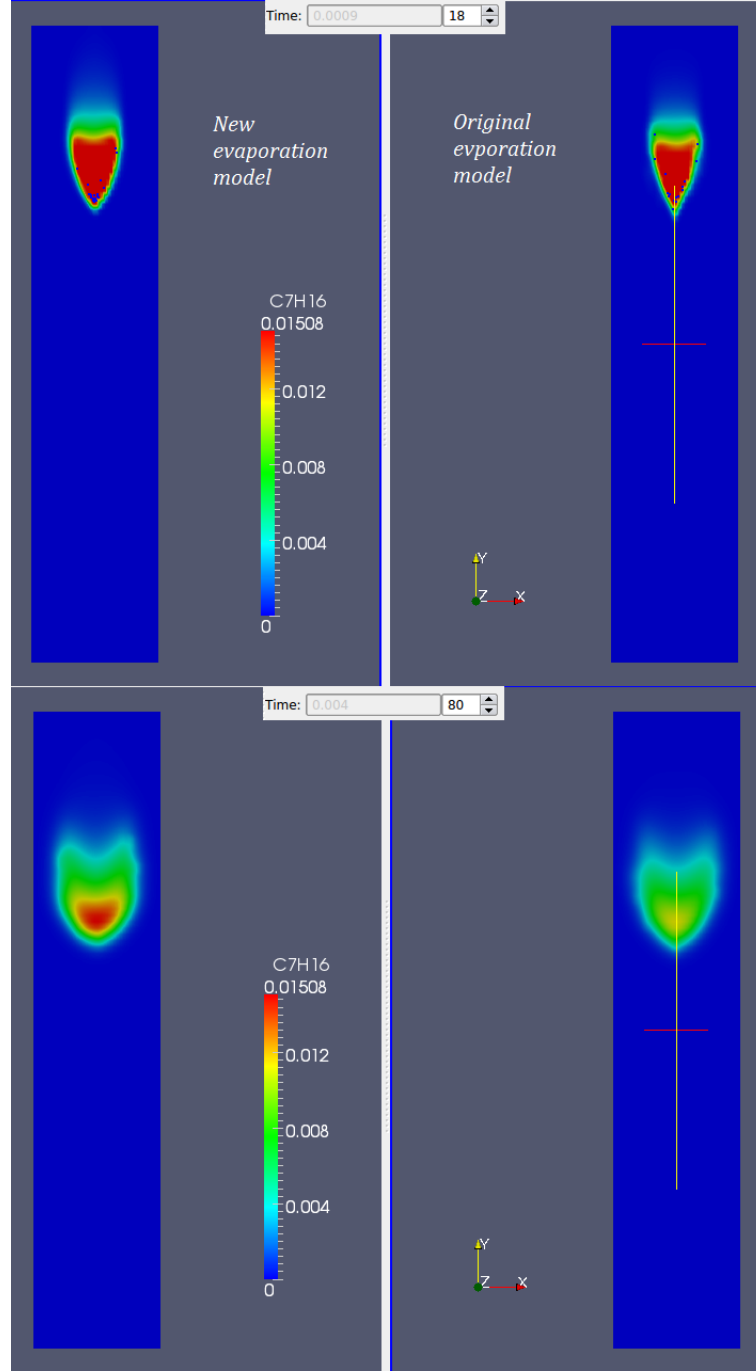


Figure 3.3: Comparison of the distribution of the vaporized specie between the two evaporation models

3.3.4 Case 4: Drag coefficient modeling

The analysis of the different formulations for drag coefficient modeling considers both:

- the solution of the equation for droplet oscillation (parameter y)
- the comparison between the original and the new equations 1.14 for calculating the drag coefficient

Solution of droplet oscillation equation

In order to confirm that the new Lagrangian sub-libraries allow for solving the droplet oscillation equation, the comparison of the droplet deformation parameter y has been shown in Figure 3.4. Results are obtained by enabling a breakup model (`breakupModel ReitzDiwakar`; in file `SprayCloudProperties`) in the base case and running it with both solvers.

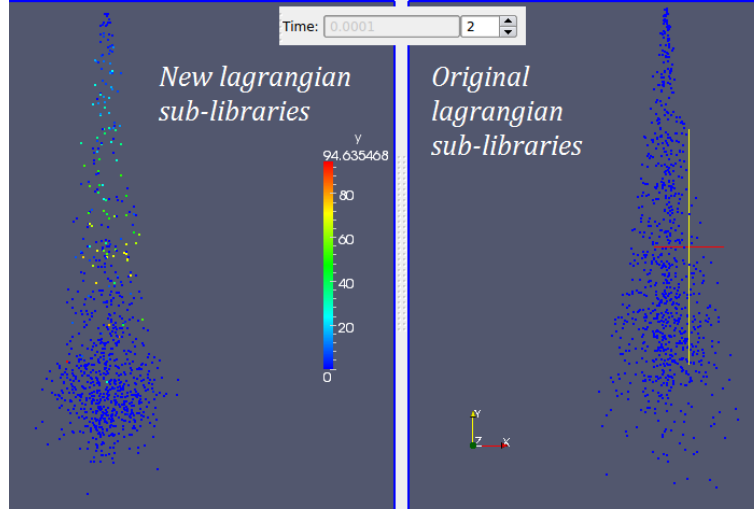


Figure 3.4: Comparison of the droplet distortion parameter y between the original and the new breakup models

Calculation of drag coefficient

Results of the two models, compared in Figure 3.5, highlights that the effect of drag coefficient calculation on the evaporation process is not significant. The similar gas-phase velocity distributions shown in Figure 3.6 are coherent, since gas is in quiet conditions, with the parcel liquid penetration. These similar results cannot be explained with a negligible difference between the original and the new formulations, but they are due to the specificity of the initial conditions of the studied case. Indeed actually the two formulations differ only for droplet Reynolds numbers between 20 and 300 (please see Eq.1.14 and the implementation in Section 2.4.1) and these values are common in the spray only after 1ms, as it can be seen in Figure 3.7. Thus, if the spray conditions are analyzed at 2ms, a significant difference between the two drag formulations can be highlighted, as Figure 3.8 reports, where the drag has been evaluated with the average spray values of Weber and Spalding heat transfer numbers.

Another confirmation of this conclusion is given by observing the evaporated mass in Figure 3.5, where a slight different evolution is visible after 1ms.

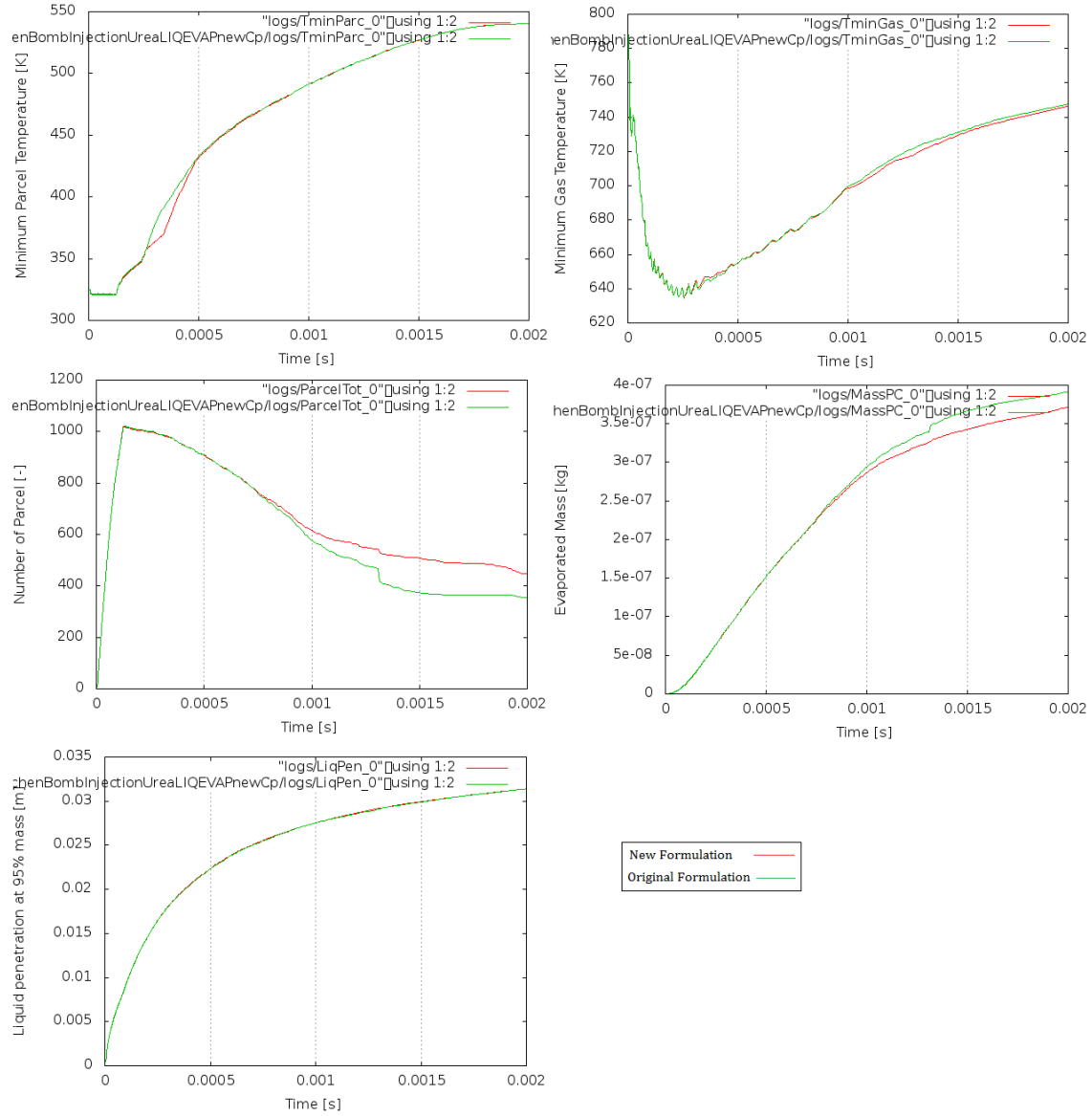


Figure 3.5: Comparison of profiles of minimum temperature of the phases, total parcel and vaporized mass in the system between the two drag coefficient models

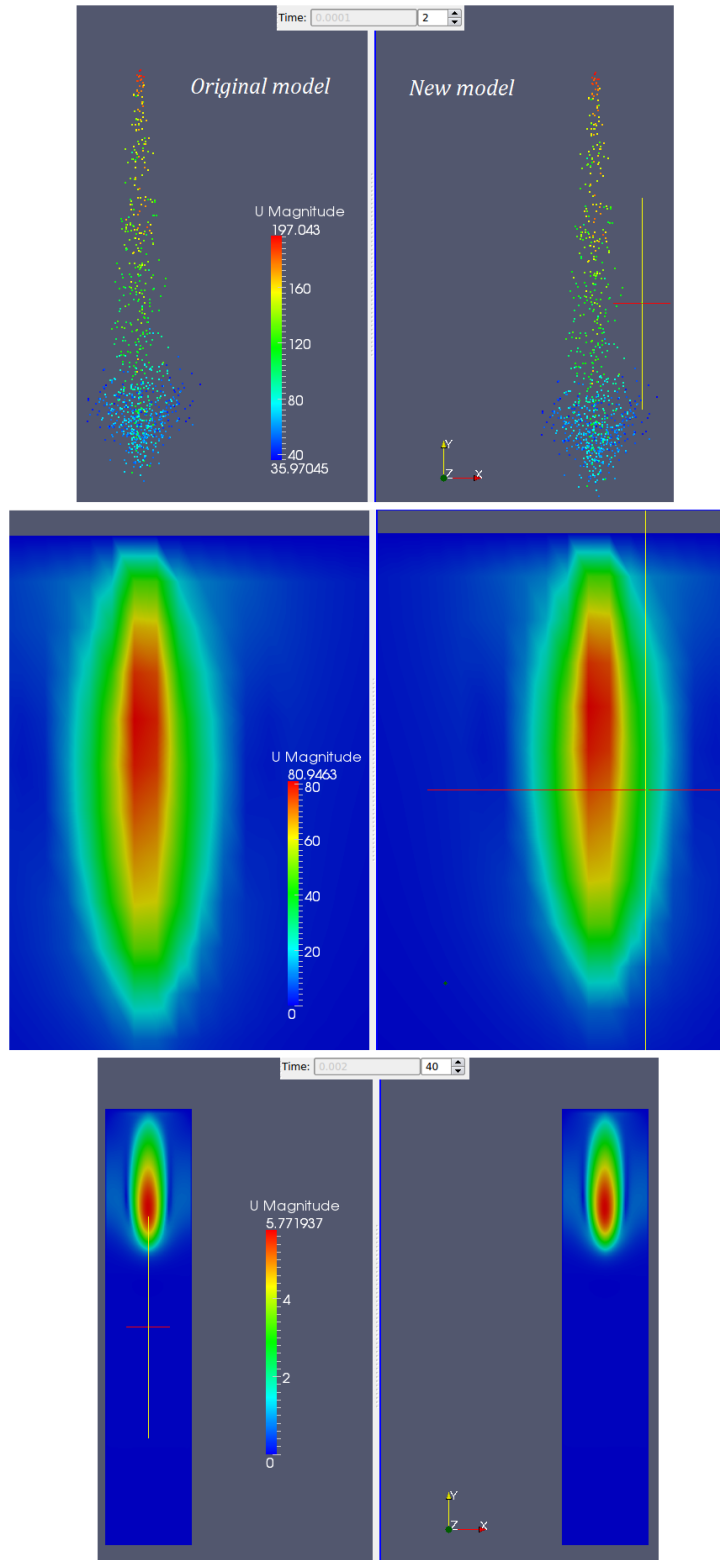


Figure 3.6: Comparison of the gas-phase velocity distributions between the two drag coefficient models

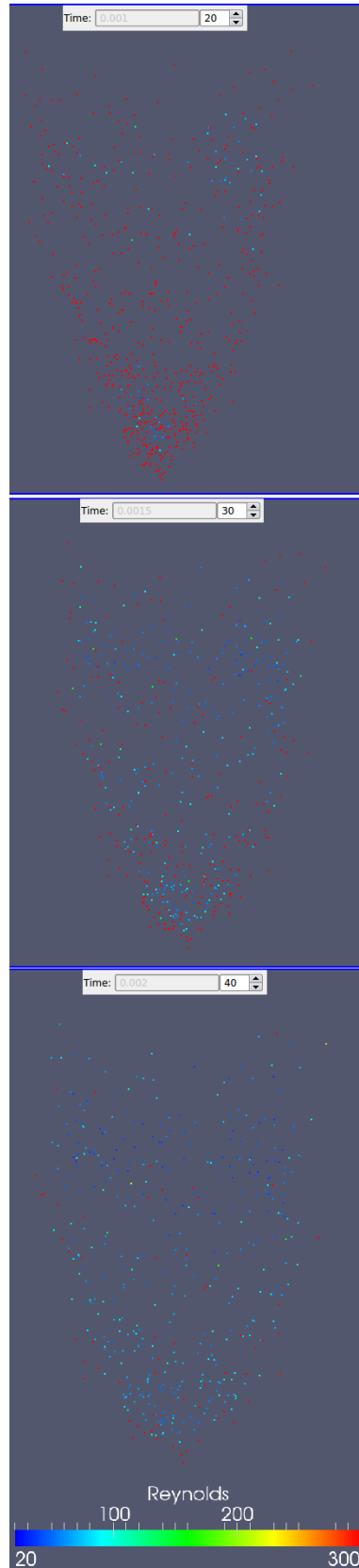


Figure 3.7: Temporal evolution of droplet Reynolds number for the solution with new formulation of drag coefficient

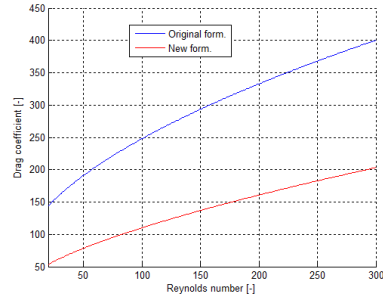


Figure 3.8: Effect of Reynolds number on drag coefficient for the two formulations by evaluating average droplets Weber and Spalding heat transfer numbers at 2ms

Bibliography

- [1] Sirignano W.A. Fluid dynamics and transport of droplets and sprays (2nd edition), Cambridge University Press, 2010
- [2] Helenbrook B.T.; Edwards C.F. Quasi-steady deformation and drag of uncontaminated liquid drops. *Int. Journ. Heat and Mass Transfer*, 28, 2002, p.1631-1657
- [3] Spalding D.B. Convective mass transfer: an introduction. Edward Arnold, 1963
- [4] Ebrahimian V.; Nicolle A.; Habchi C. Detailed modeling of the evaporation and thermal decomposition of urea-water-solution in SCR systems. *AIChE Journal*, Vol 58 Issue 7, 2012
- [5] Abramzon B.; Sirignano W.A. Droplet vaporization model for spray combustion calculations. *Int. J. Heat Mass Transfer*, Vol.32, No.9, pp.1605-1618, 1989
- [6] Shashank; Knudsen E.; Pitsch H. Spray evaporation model sensitivities. Center for Turbulence Research, Annual Research Briefs 2011
- [7] Poling B.E; Prausnitz J.M.; O'Connell J.P. The properties of gases and liquids. McGrawHill, 5th edition, 2001
- [8] Putnam A. Integrable form of droplet drag coefficient. *ARS Journal*. 31, 1961, p.1467-1468
- [9] Schiller L.; Naumann A. *VDI Zeits.* p.318, 1933
- [10] Yuen M.C.; Chen L.W. On drag of evaporating liquid droplets. *Combustion Sci. Technol.* 14, p.147-154, 1976
- [11] Warnica W.D; Renksizbulut M.; Strong A.B. Drag coefficients of spherical liquid droplets. *Experiments in Fluids*, 1994
- [12] Liu A.; Mather D.; Reitz R. Modeling the Effects of Drop Drag and Breakup on Fuel Sprays. SAE Technical Paper 930072, 1993

Nomenclature

λ	Temperature
μ	Dynamic viscosity
ϕ	Iterative factor for B_T
ρ	Density
B_M	Spalding mass transfer number
B_T	Spalding heat transfer number
c	Subscript for carrier phase
c_p	specific heat capacity
D	Binary diffusion coefficient
d	Droplet diameter
dH	Enthalpy variation due to vaporization
dM	Evaporated mass at each time step
F_M	Film thickness correction factor for mass transfer
Gr	Grashof number
H_{vap}	Vaporization Latent Heat
liq	Subscript for liquid phase (i.e. droplet)
N	Total species of carrier phase
Nu	Modified Nusselt number
Nu_0	Nusselt number
p	Pressure
$p_{sat,i}$	Vapor pressure of specie i -th
PM	Molecular weight
Pr	Prandtl number
Re	Reynolds number
ref	Subscript for film region
Sc	Schmidt number

Sh	Modified Sherwood number
Sh_0	Sherwood number
T	Temperature
u	Droplet velocity
u_c	Carrier phase velocity
$_{vap}$	Subscript for vapor phase
We	Weber number
X	Mole fraction
Y	Mass fraction

Appendices

Appendix A

Evaporation Modeling

NewLiquidEvaporation.H

```
#ifndef NewLiquidEvaporation_H
#define NewLiquidEvaporation_H

#include "PhaseChangeModel.H"
#include "liquidMixtureProperties.H"

// * * * * *

namespace Foam
{
/*-----*\
Class NewLiquidEvaporation Declaration
\*-----*/

template<class CloudType>
class NewLiquidEvaporation
:
public PhaseChangeModel<CloudType>
{
protected:

// Protected data

//- Global liquid properties data
const liquidMixtureProperties& liquids_;

//- List of active liquid names
List<word> activeLiquids_;

//- Mapping between liquid and carrier species
List<label> liqToCarrierMap_;

//- Mapping between local and global liquid species
List<label> liqToLiqMap_;

// Protected Member Functions

//- Sherwood number as a function of Reynolds and Schmidt numbers
```

```

scalar Sh(const scalar Re, const scalar Sc, const scalar Gr) const;

//- Nusselt number as a function of Reynolds and Prandtl numbers
scalar Nusselt(const scalar Re, const scalar Pr, const scalar Gr) const;

//- Calculate the carrier phase component volume fractions at cellI
tmp<scalarField> calcXc(const label cellI) const;

//- Calculate the carrier phase molecular weight at cellI
tmp<scalar> calcW(const label cellI) const;

public:

//- Runtime type information
TypeName("newLiquidEvaporation");

// Constructors

//- Construct from dictionary
NewLiquidEvaporation(const dictionary& dict, CloudType& cloud);

//- Construct copy
NewLiquidEvaporation(const NewLiquidEvaporation<CloudType>& pcm);

//- Construct and return a clone
virtual autoPtr<PhaseChangeModel<CloudType> > clone() const
{
return autoPtr<PhaseChangeModel<CloudType> >
(
new NewLiquidEvaporation<CloudType>(*this)
);
}

//- Destructor
virtual ~NewLiquidEvaporation();

// Member Functions

//- Update model
virtual void calculate
(
const scalar dt,//
const label cellI,//
scalar& Re,//
const scalar Pr,
const scalar d,//
const scalar nu,
const scalar T,//
const scalar Ts,//
const scalar pc,//
const scalar Tc,//
const scalarField& Yl,//
scalarField& dMassPC,//
scalar& bt,//

```

```

const scalar mu
) const;

//- Return the enthalpy per unit mass
virtual scalar dh
(
const label idc,
const label idl,
const scalar p,
const scalar T
) const;

//- Return vapourisation temperature
virtual scalar Tvp(const scalarField& Y) const;

//- Return maximum/limiting temperature
virtual scalar TMax(const scalar p, const scalarField& Y) const;
};
// * * * * *
} // End namespace Foam
// * * * * *
#ifdef NoRepository
# include "NewLiquidEvaporation.C"
#endif
// * * * * *
#endif

```

NewLiquidEvaporation.C

```

#include "NewLiquidEvaporation.H"
#include "specie.H"
#include "mathematicalConstants.H"

using namespace Foam::constant::mathematical;

// * * * * * Private Member Functions * * * * *

template<class CloudType>
Foam::tmp<Foam::scalarField> Foam::NewLiquidEvaporation<CloudType>::calcXc
(
const label cellI
) const
{
....
}

template<class CloudType>
Foam::scalar Foam::NewLiquidEvaporation<CloudType>::Sh
(
const scalar Re,
const scalar Sc,
const scalar Gr
) const
{
// return 2.0 + 0.552*Foam::sqrt(Re)*cbrt(Sc); //Shashank,Strom

```

```

return 2.0009 + 0.514*Foam::sqrt(max(Re,sqrt(max(Gr,0))))*cbrrt(Sc);
//Trujillo,Ebraihimian
}

template<class CloudType>
Foam::scalar Foam::NewLiquidEvaporation<CloudType>::Nusselt
(
    const scalar Re,
    const scalar Pr,
    const scalar Gr
) const
{
    // return 2.0 + 0.552*Foam::sqrt(Re)*cbrrt(Pr);
    return 2.0009 + 0.514*Foam::sqrt(max(Re,sqrt(max(Gr,0))))*cbrrt(Pr);
}

// * * * * * Constructors * * * * * //

template<class CloudType>
Foam::NewLiquidEvaporation<CloudType>::NewLiquidEvaporation
(
    const dictionary& dict,
    CloudType& owner
)
{
    .....
}

template<class CloudType>
Foam::NewLiquidEvaporation<CloudType>::NewLiquidEvaporation
(
    const NewLiquidEvaporation<CloudType>& pcm
)
{
    ....
}

// * * * * * Destructor * * * * * //

template<class CloudType>
Foam::NewLiquidEvaporation<CloudType>::~NewLiquidEvaporation()
{}

// * * * * * Member Functions * * * * * //

template<class CloudType>
void Foam::NewLiquidEvaporation<CloudType>::calculate
(
    const scalar dt,
    const label cellI,
    scalar& Re,
    const scalar Pr,
    const scalar d,
    const scalar nu,
    const scalar T,
    const scalar Ts,

```

```

const scalar pc,
const scalar Tc,
const scalarField& Yl,
scalarField& dMassPC,
scalar& bt,
const scalar mu
) const
{
// GAS PHASE carrier properties
const scalarField XcMix(calcXc(cellI));
scalarField YcMix(this->owner().thermo().carrier().Y().size(), 0.0);
scalar Wc = 0.0;
scalar Cpc = 0.0;
scalar kappac = 0.0;
scalar muc = 0.0;
forAll(this->owner().thermo().carrier().Y(), i)
{
YcMix[i] = this->owner().thermo().carrier().Y()[i][cellI];
scalar Wi = this->owner().thermo().carrier().W(i);
scalar kappai = this->owner().thermo().carrier().kappa(i, pc, Ts);
scalar mui = this->owner().thermo().carrier().mu(i, pc, Ts);
scalar kappafi = 0.0;
scalar mufi = 0.0;
forAll(this->owner().thermo().carrier().Y(), j)
{
scalar kappaj = this->owner().thermo().carrier().kappa(j, pc, Ts);
scalar muj = this->owner().thermo().carrier().mu(j, pc, Ts);
scalar Wj = this->owner().thermo().carrier().W(j);
kappafi += XcMix[j]*sqrt(1+sqrt(kappai/kappaj)*pow(Wi/Wj,0.25))/
/sqrt(8*(1+Wi/Wj));
mufi += XcMix[j]*sqrt(1+sqrt(mui/muj)*pow(Wi/Wj,0.25))/sqrt(8*(1+Wi/Wj));
} //Mason&Saxena (Poling)
kappac += (XcMix[i]*kappai)/kappafi;
muc += (XcMix[i]*mui)/mufi;
Wc += XcMix[i]*Wi;
Cpc += YcMix[i]*this->owner().thermo().carrier().Cp(i, pc, Ts);
}
scalar rhoc = pc/(specie::RR*Ts/Wc);
scalar nuc = muc/rhoc;

const scalarField X(liquids_.X(Yl));
if ((liquids_.Tc(X) - T) < SMALL)
{
if (debug)
{
....
}
forAll(activeLiquids_, i)
{
const label lid = liqToLiqMap_[i];
dMassPC[lid] = GREAT;
}
}
return;
}

```

```

forAll(activeLiquids_, i)
{
const label gid = liqToCarrierMap_[i];
const label lid = liqToLiqMap_[i];
const scalar TBoil = liquids_.properties()[lid].pvInvert(pc);
const scalar Tfilm = min(Ts, 0.999*TBoil);

// Concentration of the ACTIVE LIQUID in CARRIER phase
const scalar Xc = XcMix[gid];
const scalar Yc = YcMix[gid]; // carrier phase concentration

// VAPOR properties (ACTIVE liquids)
const scalar pSat0 = liquids_.properties()[lid].pv(pc, T);
//Ebrahimian,Ramadan,Shashank,Sirignano at T
const scalar pVap = X[lid]*pSat0;
const scalar CpVap = liquids_.properties()[lid].Cpg(pVap, Tfilm);
// specific heat capacity for lid - Shashank measured at Tfilm
const scalar rhoVap = pVap*liquids_.properties()[lid].W()/(specie::RR*Tfilm);
const scalar muVap = liquids_.properties()[lid].mug(pVap, Tfilm);
// vapour dynamic viscosity - Shashank measured at Tfilm
const scalar kappaVap = liquids_.properties()[lid].Kg(pVap, Tfilm);
// vapour thermal conductivity - Shashank measured at Tfilm
const scalar WVap = liquids_.properties()[lid].W();
const scalar DabVap = liquids_.properties()[lid].D(pVap, Tfilm, Wc);
// vapour diffusivity in air [m2/s] - Shashank measured at Tfilm

const scalar Xs = X[lid]*pSat0/pc;
// surface VAPOUR molar fraction - Raoult's Law
const scalar Ys = WVap*Xs/(WVap*Xs + Wc*(1 - Xs));
//assuming at FILM only VAPOUR (Xs) & "freestream" gas (1-Xs)

// REFERENCE (FILM) COMPOSITION
const scalar Xfilm = (2*Xs + Xc)/3; //Shanshank
const scalar Yfilm = (2*Ys + Yc)/3;

const scalar CpFilm = CpVap*Yfilm + Cpc*(1 - Yfilm);
//Shashank,Ramadan,Sirignano
const scalar rhoFilm = 1/(Yfilm/rhoVap + (1 - Yfilm)/rhoC); //Sirignano
const scalar lfifg = sqrt(1+sqrt(kappaVap/kappac)*pow(Wc/WVap,0.25))/
/sqrt(8*(1+WVap/Wc)); //Shashank,Ramadan (Mason)
const scalar lfigf = sqrt(1+sqrt(kappac/kappaVap)*pow(WVap/Wc,0.25))/
/sqrt(8*(1+Wc/WVap)); //Shashank,Ramadan (Mason)
const scalar kappaFilm = kappaVap*Xfilm/(Xfilm + (1 - Xfilm)*lfifg) +
+ kappac*(1 - Xfilm)/(1 - Xfilm + Xfilm*lfigf); //Shashank,Ramadan(Mason)

// const scalar muFilm = muVap*Xfilm/(Xfilm + (1 - Xfilm)*sqrt(Wc/WVap)) +
+ muc*(1 - Xfilm)/(1 - Xfilm + Xfilm*sqrt(WVap/Wc)); //Strom (Zipperer)
const scalar mfifg = sqrt(1+sqrt(muVap/muc)*pow(Wc/WVap,0.25))/
/sqrt(8*(1+WVap/Wc)); //Shashank,Ramadan (Wilke)
const scalar mfigf = sqrt(1+sqrt(muc/muVap)*pow(WVap/Wc,0.25))/
/sqrt(8*(1+Wc/WVap)); //Shashank,Ramadan (Wilke)
const scalar muFilm = muVap*Xfilm/(Xfilm + (1 - Xfilm)*mfifg) +
+ muc*(1 - Xfilm)/(1 - Xfilm + Xfilm*mfigf); //Shashank,Ramadan (Wilke)

```

```

const scalar nuFilm = muFilm/rhoFilm;

// TRANSFER PARAMETERS
const scalar ScFilm = nuFilm/(DabVap + ROOTVSMALL); // Ebrahimian
//      const scalar ScFilm = nuc/(Dab + ROOTVSMALL); // only Gas
const scalar Gr = 9.81*pow(d,3)*(Tc-T)/sqr(nuc)/Tc; //Trujillo,Ebrahimian
//      const scalar Gr = 0; //altri
Re = Re*mu/muFilm; //Strom,Sirignano
//      const scalar Prc = muFilm*CpFilm/kappac; //Shashank
const scalar Prc = muFilm*CpFilm/kappaFilm; //Ramadan
const scalar Le = ScFilm/Prc; // Lewis Number
const scalar Nu0 = this->Nusselt(Re, Prc, Gr);
//only gas phase for Nusselt (Ramadan)
const scalar Sh0 = this->Sh(Re, ScFilm, Gr); // Sherwood number
const scalar Bm = (Ys - Yc)/max(SMALL, 1.0 - Ys); // mass transfer number
scalar Nu = Nu0;
scalar Sh = Sh0;
if (Bm > 0)
{
const scalar Fm = pow((1 + Bm), 0.7)/Bm*log(1 + Bm);
// mass transfer correction factor
//      const scalar Fm = pow((1 + Bm), 0.7); // Ramadan
//      const scalar Fm = 1; // Shashank
Sh = 2 + (Sh0-2)/Fm; // Strom - corrected Sherwood
//      const scalar Sh = Sh0/Fm; // Ramadan,Shashank

//Iteration for Bt:
scalar Ft = 0.0; // heat transfer correction factor
Nu = 0.0; // corrected Nusselt
scalar psiB = 0.0; // Exponent
scalar eps = 1.0; // Relative Error
scalar Btiter = bt; //Bt of previous timestep
for (label iter = 0; iter < 5; iter++)
{
if (eps > 5e-2)
{
Ft = pow((1 + Btiter), 0.7)/Btiter*log(1 + Btiter); //Strom
//      Ft = pow((1 + Btiter), 0.7); //Ramadan
//      Ft = 1; //Shashank
Nu = 2 + (Nu0-2)/Ft; //Strom
//      Nu = Nu0/Ft; //Ramadan,Shashank
psiB = CpVap/CpFilm*Sh/Nu/Le; //Strom,Shashank
eps = std::abs((pow((1 + Bm), psiB)-1)/Btiter-1);
Btiter = (pow((1 + Bm), psiB)-1);
}
}
bt = Btiter;
}
if (Xc*pc > pSat0)
{
// saturated vapour - no phase change
}
else
{

```

```

if (Xs > 0.999)
{ // boiling
dMassPC[lid] = pi*DabVap*kappac*Nu/CpVap*log(1.0 + bt)*dt;
// Birkhold
}
else
{ // evaporation
dMassPC[lid] = pi*d*Sh*DabVap*rhoFilm*dt*log(1.0 + Bm);
// mass transfer [kg] - Strom,Shashank
}
}
}
}

template<class CloudType>
Foam::scalar Foam::NewLiquidEvaporation<CloudType>::dh
(
const label idc,
const label idl,
const scalar p,
const scalar T
) const
{
....
}

template<class CloudType>
Foam::scalar Foam::NewLiquidEvaporation<CloudType>::Tvap
(
const scalarField& Y
) const
{
....
}

template<class CloudType>
Foam::scalar Foam::NewLiquidEvaporation<CloudType>::TMax
(
const scalar p,
const scalarField& Y
) const
{
....
}

// ***** //

```

Appendix B

Drag Coefficient

DeformedSphereDrag.H

```
#ifndef DeformedSphereDrag_H
#define DeformedSphereDrag_H

#include "ParticleForce.H"

// * * * * *

namespace Foam
{
/*-----*\
Class DeformedSphereDrag Declaration
\*-----*/

template<class CloudType>
class DeformedSphereDrag
:
public ParticleForce<CloudType>
{
// Private Member Functions

//- Drag coefficient multiplied by Reynolds number
scalar CdRe(const scalar Re, const scalar y, const scalar YPhase,
const scalar Bt, const scalar We) const;

public:

//- Runtime type information
TypeName("deformedSphereDrag");

// Constructors
//- Construct from mesh
DeformedSphereDrag
(
CloudType& owner,
const fvMesh& mesh,
const dictionary& dict
);
```

```

//- Construct copy
DeformedSphereDrag(const DeformedSphereDrag<CloudType>& df);

//- Construct and return a clone
virtual autoPtr<ParticleForce<CloudType> > clone() const
{
return autoPtr<ParticleForce<CloudType> >
(
new DeformedSphereDrag<CloudType>(*this)
);
}

//- Destructor
virtual ~DeformedSphereDrag();

// Member Functions

// Evaluation

//- Calculate the coupled force
virtual forceSuSp calcCoupled
(
const typename CloudType::parcelType& p,
const scalar dt,
const scalar mass,
const scalar Re,
const scalar muc,
const scalar y,
const scalar YPhase,
const scalar Bt,
const scalar We
) const;
};

// * * * * *
} // End namespace Foam
// * * * * *
#ifdef NoRepository
# include "DeformedSphereDrag.C"
#endif
// * * * * *
#endif

DeformedSphereDrag.C

#include "DeformedSphereDrag.H"

// * * * * * Private Member Functions * * * * *

template<class CloudType>
Foam::scalar Foam::DeformedSphereDrag<CloudType>::CdRe
(
const scalar Re,
const scalar y,

```

```

const scalar YPhase,
const scalar Bt,
const scalar We) const
{

scalar Cd = 0;
scalar bit = Bt;
if (bit==2) //i.e. no evaporation
{
bit = 0;
}

if ((YPhase - 1.0) < SMALL)
{
if (Re > 1000.0)
{
Cd = 0.424*Re; //Kiva
//      Cd = 0.44*Re; //classico
}
else if (Re <= 20.0)
{
Cd = 24.0*(1.0 + 0.13*pow(Re, (0.82-log10(Re)))); //Clift
}
else if (20 < Re && Re < 300.0)
{
//      Cd = 24.0*(1.0 + 1.0/6.0*pow(Re, 2.0/3.0)); //classico Yuen
Cd = 24.0/pow((1+bit), 0.2)*(1.0 + 0.2*pow(Re, 0.63))*
*(1+0.06*pow(Re, -0.12)*pow(We, 1.4)); //Renksizbulut
}
else
{
Cd = 24.0*(1.0 + 1.0/6.0*pow(Re, 2.0/3.0)); //Yuen (better, ref.Kralj)
//      Cd = 24.0*(1.0 + 0.15*pow(Re, 0.687)); //Clift,Fritsching
}
return Cd;
//      return Cd*(1+2.632*min(1.0,y)); //Reitz,Sirignano
//((ALTERNATIVE to Renksizbulut!!!!!!))
}
else
{
if (Re > 1000.0)
{
Cd = 0.44*Re;
}
else
{
Cd = 24.0*(1.0 + 1.0/6.0*pow(Re, 2.0/3.0)); //Yuen (better, ref.Kralj)
//      Cd = 24.0*(1.0 + 0.15*pow(Re, 0.687)); //Clift,Fritsching
}
return Cd;
}
}

// * * * * * Constructors * * * * *

```

```

template<class CloudType>
Foam::DeformedSphereDrag<CloudType>::DeformedSphereDrag
(
    CloudType& owner,
    const fvMesh& mesh,
    const dictionary& dict
)
:
ParticleForce<CloudType>(owner, mesh, dict, typeName, false)
{}

template<class CloudType>
Foam::DeformedSphereDrag<CloudType>::DeformedSphereDrag
(
    const DeformedSphereDrag<CloudType>& df
)
:
ParticleForce<CloudType>(df)
{}

// * * * * * D e s t r u c t o r * * * * * //

template<class CloudType>
Foam::DeformedSphereDrag<CloudType>::~DeformedSphereDrag()
{}

// * * * * * M e m b e r   F u n c t i o n s   * * * * * //

template<class CloudType>
Foam::forceSuSp Foam::DeformedSphereDrag<CloudType>::calcCoupled
(
    const typename CloudType::parcelType& p,
    const scalar dt,
    const scalar mass,
    const scalar Re,
    const scalar muc,
    const scalar y,
    const scalar YPhase,
    const scalar Bt,
    const scalar We
) const
{

    forceSuSp value(vector::zero, 0.0);

    value.Sp() = mass*0.75*muc*CdRe(Re, y, YPhase, Bt, We)/(p.rho()*sqr(p.d()));
    return value;
}

```

Appendix C

New Lagrangian sub-libraries and solver

Allwmake

```
#!/bin/sh
cd ${0%/*} || exit 1    # run from this directory
makeType=${1:-libso}
set -x
wmake $makeType intermediate
wmake $makeType spray
#-----end-of-file
```

Allwclean

```
#!/bin/sh
cd ${0%/*} || exit 1    # run from this directory
makeType=${1:-libso}
set -x
wclean $makeType intermediate
wclean $makeType spray
#-----end-of-file
```